

Algorithmique et Programmation 1

A. Boussidan, L. Exibard, P. Yvonnet

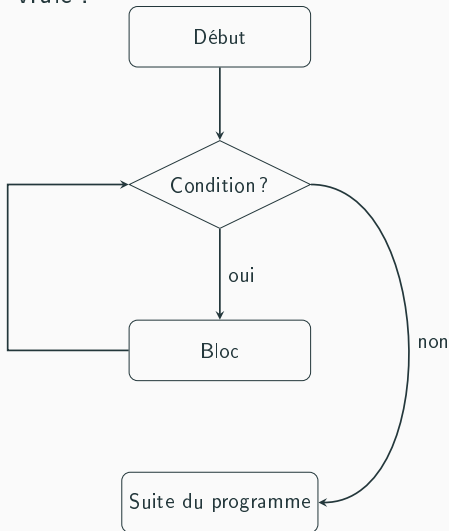
28 Septembre 2026

- C'est la fin du cycle « briques de base » !
- Les notes de cours sont disponibles sur E-learning.
- Pas de CC0...

- C'est la fin du cycle « briques de base » !
- Les notes de cours sont disponibles sur E-learning.
- Pas de CC0...
- ... Mais un TP noté la semaine prochaine !

Un retour sur les boucles while

Que faire si on veut répéter une instruction si une condition reste vraie ?



```
# début
while condition:
    # bloc v
    # (corps de la boucle)
# suite
```

Itération

Une exécution complète du corps de la boucle

Un peu de vocabulaire

Itération

Une exécution complète du corps de la boucle

Variable ou indice de boucle

La variable qui compte le nombre de tours de boucles (si elle existe).

→ Souvent notée i ou j .

Un peu de vocabulaire

Itération

Une exécution complète du corps de la boucle

Variable ou indice de boucle

La variable qui compte le nombre de tours de boucles (si elle existe).

→ Souvent notée i ou j .

Condition de continuation

La condition qui est vérifiée à chaque début de boucle : si elle est vraie, on continue.

Un peu de vocabulaire

Itération

Une exécution complète du corps de la boucle

Variable ou indice de boucle

La variable qui compte le nombre de tours de boucles (si elle existe).

→ Souvent notée i ou j .

Condition de continuation

La condition qui est vérifiée à chaque début de boucle : si elle est vraie, on continue.

Condition d'arrêt

La négation de la condition de continuation.

Sur un exemple

```
i = 0 # indice de boucle
while i <= 15: #condition de continuation
    print("Ceci est l'itération numéro", i )
    i = i + 1 #Incrémentation
# Ici, i > 15: condition d'arrêt
```

Un exemple familier...

```
from random import randint

win_L, win_T = 0, 0
courses = 0 # indice de boucle 1

while courses < 1000: # condition de continuation (boucle 1)
    pos_L = 0
    pos_T = 0
    winner = "Tortue"
    while pos_T < 6 and pos_L < 6: # condition de continuation (boucle 2)
        lancer = randint(1, 6)
        if lancer == 6:
            pos_L = 6
            win_L += 1
        else:
            pos_T += lancer
            if pos_T >= 6:
                win_T += 1
```

Un exemple familier...

```
from random import randint

win_L, win_T = 0, 0
courses = 0  # indice de boucle 1

while courses < 1000:  # condition de continuation (boucle 1)
    pos_L = 0
    pos_T = 0
    winner = "Tortue"
    while pos_T < 6 and pos_L < 6:  # condition de continuation (boucle 2)
        lancer = randint(1, 6)
        if lancer == 6:
            pos_L = 6
            win_L += 1
        else:
            pos_T += lancer
            if pos_T >= 6:
                win_T += 1
```

M'sieur, ça marche pas !

Un exemple familial... corrigé

```
from random import randint

win_L, win_T = 0, 0
courses = 0 # indice de boucle 1

while courses < 1000: # condition de continuation (boucle 1)
    pos_L = 0
    pos_T = 0
    winner = "Tortue"
    while pos_T < 6 and pos_L < 6: # condition de continuation (boucle 2)
        lancer = randint(1, 6)
        if lancer == 6:
            pos_L = 6
            win_L += 1
        else:
            pos_T += lancer
            if pos_T >= 6:
                win_T += 1
    courses += 1 # Ne pas oublier la mise à jour !
```

Attention à la boucle infinie

Comment débogguer une boucle while

- Ajouter des prints dans son code

Comment déboguer une boucle while

- Ajouter des prints dans son code
- Exécuter son code pas à pas (à la main, sur PythonTutor, ou avec le débogueur)

Comment déboguer une boucle while

- Ajouter des prints dans son code
- Exécuter son code pas à pas (à la main, sur PythonTutor, ou avec le débogueur)
- S'il y a une boucle infinie : se concentrer sur les variables du test de continuation
 - Sont-elles bien initialisées ?
 - Quand sont-elles mises à jour ?
 - Pourquoi est-ce que la boucle finit par s'arrêter ?

Boucles while - Cas d'utilisation

Rappel - être un chef!

On commence toujours par formuler l'algorithme. Si votre algorithme contient ...

Boucles while - Cas d'utilisation

Rappel - être un chef!

On commence toujours par formuler l'algorithme. Si votre algorithme contient ...

"...et on recommence"

"jusqu'à ce que..."

"s'il en reste, on refait"

"on repart du début"

Boucles while - Cas d'utilisation

Rappel - être un chef!

On commence toujours par formuler l'algorithme. Si votre algorithme contient ...

"...et on recommence"

"jusqu'à ce que..."

"s'il en reste, on refait"

"on repart du début"

Il faut une boucle while !

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

"On demande de saisir une première fois,
et on recommence tant que la saisie est incorrecte."

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

"**On demande** de saisir **une première fois**,
et **on recommence tant que** la saisie est
incorrecte."

Boucle while



Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

input

"**On demande** de saisir **une première fois**,
et **on recommence tant que** la saisie est
incorrecte."

Boucle while

Exemple - saisie contrôlée

Principe de la saisie contrôlée

Demander à l'utilisateur de saisir quelque chose et vérifier que sa saisie est correcte

Quel est l'algorithme ?

input

initialisation
avant la boucle

"**On demande** de saisir **une première fois**,
et **on recommence tant que** la saisie est
incorrecte."

Boucle while

Saisie contrôlée - Une implémentation

```
parfum = input("Choisissez un parfum de glace")

while parfum != "chocolat" and parfum != "vanille" and parfum != "fraise":
    print("Nous n'avons pas ça en stock...")
    parfum = input("Choisissez un parfum de glace")

print("Bien sûr, voici votre glace parfum " + parfum + " !")
```

Saisie contrôlée - Une implémentation

```
parfum = input("Choisissez un parfum de glace")

while parfum != "chocolat" and parfum != "vanille" and parfum != "fraise":
    print("Nous n'avons pas ça en stock...")
    parfum = input("Choisissez un parfum de glace")

print("Bien sûr, voici votre glace parfum " + parfum + " !")
```

- Comment faire pour ajouter d'autres parfums possibles ?
- Que se passe-t-il si on remplace les `and` par des `or` ?
- On est obligé de copier/coller la ligne avec le `input`

Un nouveau problème

On sait déjà répéter une instruction avec un `while`.

Comment afficher chaque lettre d'un mot, une par une?

Un nouveau problème

On sait déjà répéter une instruction avec un `while`.

Comment afficher chaque lettre d'un mot, une par une?

```
mot = "Python"
```

Indice ou élément ?

P	y	t	h	o	n
---	---	---	---	---	---

Indice ou élément ?

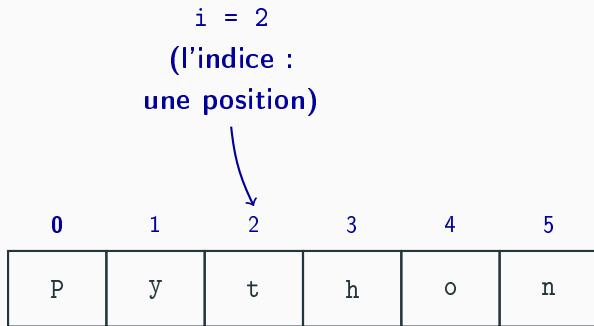
0	1	2	3	4	5
P	y	t	h	o	n

Indice ou élément ?

0	1	2	3	4	5
P	y	t	h	o	n

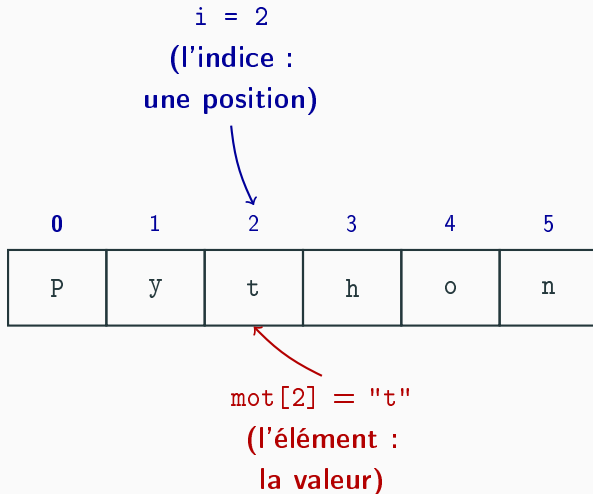
Dans une `str`, les éléments sont numérotés à partir de 0.

Indice ou élément ?



Dans une `str`, les éléments sont numérotés à partir de 0.

Indice ou élément ?



Dans une `str`, les éléments sont numérotés à partir de 0.

Quel **algorithme** utiliser pour afficher les lettres une par une ?

Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

```
mot = "Python"
position = 0
while position < len(mot):
    print(mot[position])
    position = position + 1
```

Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

```
mot = "Python"
i = 0
while i < len(mot):
    print(mot[i])
    i = i + 1
```

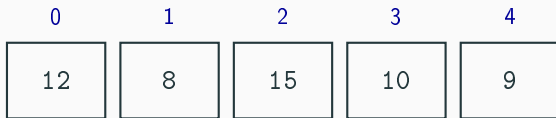
Quel **algorithme** utiliser pour afficher les lettres une par une ?

On va faire parcourir les **positions** de la chaîne, et on va afficher
l'**élément** à cette position.

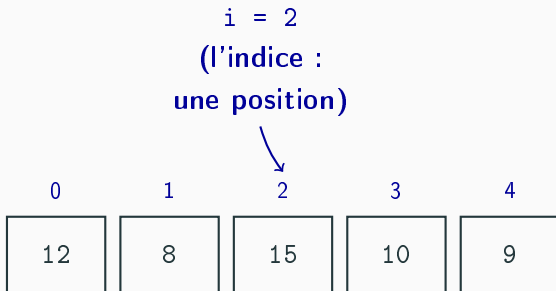
```
mot = "Python"
i = 0
while i < len(mot):
    print(mot[i])
    i = i + 1
```

- i varie de 0 à $\text{len}(\text{mot}) - 1$
- à chaque tour, on accède à `mot[i]`

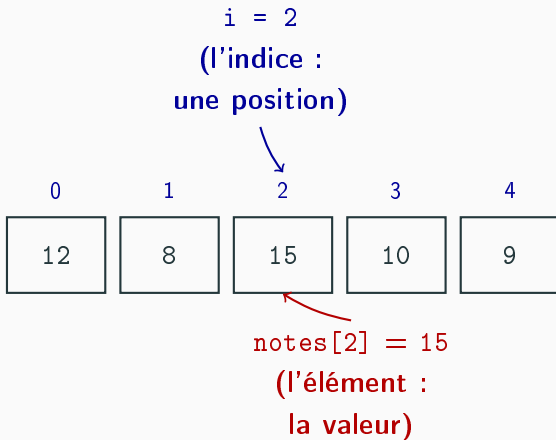
Et pour une liste ?



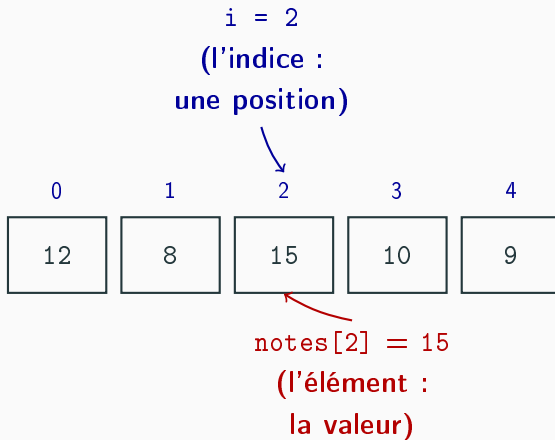
Et pour une liste ?



Et pour une liste ?



Et pour une liste ?



Ca devrait être familier...

Et pour une liste ?

Le même programme marche pour une liste !

```
notes = [12, 8, 15, 10]
```

```
i = 0
```

```
while i < len(notes):
```

```
    print(notes[i])
```

```
    i = i + 1
```

Et pour une liste ?

Le même programme marche pour une liste !

```
notes = [12, 8, 15, 10]
i = 0
while i < len(notes):
    print(notes[i])
    i = i + 1
```

Coïncidence ? ...

Et pour une liste ?

Le même programme marche pour une liste !

```
notes = [12, 8, 15, 10]
i = 0
while i < len(notes):
    print(notes[i])
    i = i + 1
```

Coïncidence ? ...

Non, itérables !

Un **itérable** est un objet qui regroupe plusieurs valeurs, que l'on peut parcourir élément par élément.

En Python, les itérables les plus courants sont :

- `str`
- `list`
- `tuple` (vu plus tard)
- `dict` (vu plus tard)

Un **itérable** est un objet qui regroupe plusieurs valeurs, que l'on peut parcourir élément par élément.

En Python, les itérables les plus courants sont :

- `str`
- `list`
- `tuple` (vu plus tard)
- `dict` (vu plus tard)
- `range`

```
range(a, b)
```

génère les entiers de a (inclus) à b (exclu)

```
range(a, b)
```

génère les entiers de a (inclus) à b (exclu)

```
range(0, 6)
```

L'itérable range

`range(a, b)`

génère les entiers de a (inclus) à b (exclu)

`range(0, 6)`

0	1	2	3	4	5
---	---	---	---	---	---

Un outil pour parcourir les itérables : la boucle `for` !

Un outil pour parcourir les itérables : la boucle `for` !

À retenir

La boucle `for` parcourt toujours les **éléments** d'un itérable.

Un outil pour parcourir les itérables : la boucle for !

À retenir

La boucle for parcourt toujours les éléments d'un itérable.

str

```
mot = "Python"  
for element in mot:  
    print(element)
```

list

```
notes = [12, 8, 15, 10, 9]  
for element in notes:  
    print(element)
```

La boucle for avec range

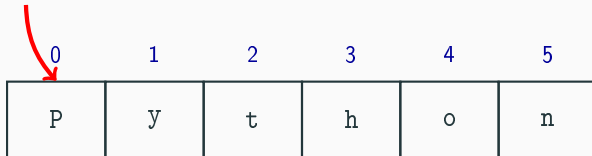
```
for i in range(0, 6):  
    print(i)
```

La boucle for avec range

```
for i in range(0, 6):  
    print(i)
```

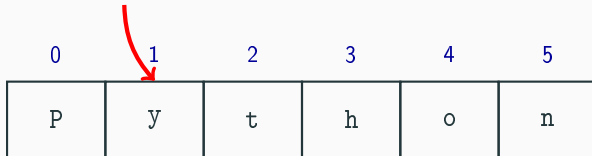
i prend successivement chaque **élément** de range(0, 6) :
0, 1, 2, 3, 4, 5.

La boucle for sur une str



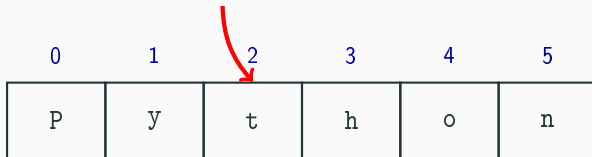
```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une str



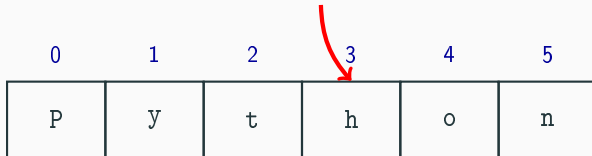
```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une str



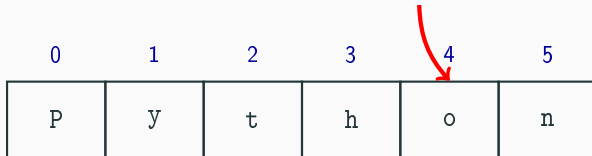
```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une str



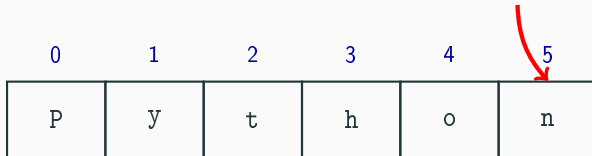
```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une str



```
mot = "Python"  
for element in mot:  
    print(element)
```

La boucle for sur une str



```
mot = "Python"  
for element in mot:  
    print(element)
```